

Lean Formal Proof Results

ARM AArch64 Virtualisation Architecture

EL2 hypervisor configuration, Stage-2 memory translation, and vCPU lifecycle management on Microsoft Cobalt (Neoverse N2/V3)

Document identifier	A64-CBLT-STUDY-001
Version / date	1.0 / 12 August 2026
Study type	Formal proof results and architecture/API specification review
Author	Dhruv Setty
Primary baseline	Arm DDI0487K and DDI0608A; Neoverse N2/V3; Linux KVM/arm64; QEMU target/arm/
Study artefact	https://github.com/whatheheckisthis/Intent-to-Auditable-Trust-Object-v7
Companion input	prism-uploads/Excerpt_arXiv.pdf
Lean artefact	FormalResults.lean
Target substrate	Azure Cobalt 100 / Cobalt 200; Arm Neoverse N2 / V3 family

Purpose. Produce an auditable engineering judgement and a Lean 4 formalisation of the quotient, refinement, migration, Stage-2 permission, EL2-profile, and KVM/arm64 lifecycle results. Findings are classified as *Correction*, *Precision*, *Conforms*, or *Verification gap*. Lean proves consequences of the declared model; it does not certify a particular Azure host, kernel, QEMU build, firmware image, or Arm implementation.

Distribution: technical review; systems security; hypervisor engineering; verification and assurance.

Document Control and Abstract

Abstract

This study reviews and formalises a proposed execution model for AArch64 virtual machines on Cobalt-class Arm servers. The six structural derivations and closure theorem in the supplied four-page excerpt are encoded in `FormalResults.lean`. Small executable proof models then capture Stage-2 permission composition, profile-scoped HCR_EL2.TGE, and the SVE-enabled KVM vCPU lifecycle. Engineering premises are checked against Arm DDI0487K, DDI0608A, the Neoverse N2 r0p1 and V3/CSS V3 materials, Linux KVM/arm64 documentation, and QEMU `target/arm/`. Platform observations remain explicit test obligations rather than logical axioms.

The proof results establish observational equivalence as an equivalence relation, refinement as a pulled-back preorder, observation-preserving embedding and migration identity, bidirectional representative independence under transition congruence, Stage-2 denial dominance, non-universality of TGE=0, and the absence of a legal configured-to-running SVE transition. The engineering review retains two conformance-blocking corrections and two precision defects: Cobalt 100 is Neoverse N2 / Armv9.0-A; SVE finalisation is a distinct gate; the VLS write selects but does not lock; and TGE=0 is scoped to the selected non-VHE profile.

Review states

Class	Effect	Decision rule
Correction	Blocks conformance	A factual or transition error that must be repaired.
Precision	Conditional conformance	Intent is defensible, but scope or mechanism is ambiguous.
Conforms	Accepted	Claim is supported within the stated design profile.
Verification gap	Evidence required	Plausible claim depends on a concrete host, kernel, firmware, or measurement.

Change and approval record

Version	Date	Owner	Change / decision
1.0	12 Aug 2026	Dhruv Setty	Initial 22-page formal proof results and technical specification.

Contents

Document Control and Abstract	2
1 Executive Results	4
2 Scope, Questions, and Method	5
3 Architecture Baseline and Selected Profile	6
4 EL2 Hypervisor Configuration Results	7
5 EL2 Context, Entry, and Exit	8
6 Stage-2 Translation Model	9
7 VTTBR, VTCR, and Address-Space Configuration	10
8 Stage-2 Updates, VMIDs, and TLB Maintenance	11
9 Memory Types, Permissions, and Fault Handling	12
10 KVM/arm64 API Object and Capability Model	13
11 vCPU Lifecycle State Machine	14
12 SVE/SVE2 Configuration and Finalisation	15
13 Migration and Observable Equivalence	16
14 Lean Formal Proof Results	17
15 Cobalt N2/V3 Substrate Assessment	19
16 Security and Microarchitectural Review	20
17 Verification Matrix and Result Schema	21
18 Conclusions, Actions, and References	22

Glossary and Notation

Term	Meaning	Term	Meaning
EL2	AArch64 hypervisor exception level.	IPA	Intermediate physical address produced by guest Stage-1 translation.
Stage-2	EL2-controlled IPA-to-PA translation and permission regime.	PA	Physical address selected after translation.
KVM	Linux Kernel-based Virtual Machine interface.	VMM	Userspace virtual-machine monitor using KVM.
vCPU	Virtual processor represented by a KVM vCPU object.	VMID	Identifier tagging a live Stage-2 translation context.
VHE	Virtualisation Host Extensions; permits an EL2-hosted kernel model.	non-VHE	Profile with the guest at EL1 and separately scoped EL2 host controls.
SVE	Scalable Vector Extension architectural state.	VLS	Set of vector lengths exposed to an SVE-enabled vCPU.
TLB/TLBI	Translation cache and its invalidation operations.	MMIO	Memory-mapped device input/output aperture.
VA	Guest virtual address before Stage-1 translation.	Trust object	Auditable claim linked to source, scope, observation, and test evidence.

Reading convention. *Shall* denotes a reviewed design requirement. *Conforms* denotes specification consistency, not implementation certification. Register names and ioctl symbols are rendered literally.

1 Executive Results

1.1 Overall disposition

The proposed model is **conditionally conformant after correction**. Stage-2 isolation, the principal EL2 register set, exit handling, SVE architectural state, and the high-level microarchitectural threat model are coherent. The two Correction findings affect platform identity and executable lifecycle semantics; both are straightforward to repair but must not be waived.

ID	Class	Area	Result
F-01	Correction	Cobalt 100	Replace Armv8.5-A with Armv9.0-A; retain Neoverse N2.
F-02	Correction	vCPU lifecycle	Insert <i>finalised</i> before first KVM_RUN; migration restores to an equivalent finalised state.
F-03	Precision	SVE VL	SET_ONE_REG selects VLS; VCPU_FINALIZE locks the selection.
F-04	Precision	EL2 traps	Scope TGE=0 to the chosen non-VHE profile; VHE permits other valid configurations.
F-05	Verification gap	Cobalt 200	Validate V3/CSS V3 feature exposure from host ID registers and Azure SKU evidence.
F-06	Verification gap	VMID/TLB	Demonstrate required invalidation sequence under VMID reuse and migration.
F-07	Verification gap	Memory types	Prove that guest RAM and MMIO Stage-2 attributes match the actual device map.
F-08	Verification gap	Side channels	Record firmware/kernel mitigations and measurable residual risk per host generation.

1.2 Conformance by study domain

Domain	Disposition	Finding
EL2 hypervisor control	Conforms with precision	F-04
Stage-2 translation and isolation	Conforms; test evidence required	F-06, F-07
KVM arm64 vCPU lifecycle	Non-conformant until corrected	F-02, F-03
Cobalt substrate description	Mixed; one correction, one gap	F-01, F-05
Migration and observable equivalence	Conforms as a model; prove in implementation	F-06
Microarchitectural controls	Plausible baseline; host evidence required	F-08

1.3 Release recommendation

Revise the four repository findings, add capability discovery and negative lifecycle tests, then permit implementation prototyping. Do not claim Cobalt-wide portability from an N2-only result, or V3 feature availability without SKU-level evidence.

2 Scope, Questions, and Method

2.1 Study questions

1. Does the selected EL2 profile establish a coherent trap, exception, and execution regime for an EL1 guest?
2. Do VTTBR_EL2, VTCR_EL2, Stage-2 descriptors, VMIDs, and TLB maintenance support the asserted isolation boundary?
3. Is the proposed KVM/arm64 vCPU state machine legal, including SVE/SVE2 configuration and migration?
4. Which claims are architectural, which are Linux ABI obligations, and which depend on Cobalt host implementation?
5. Can every accepted claim be represented as an auditable trust object with source, scope, observation, and test evidence?

2.2 Evidence hierarchy

Rank	Source class	Use
1	Arm architecture specification	Register semantics, exception levels, translations, maintenance requirements.
2	Linux KVM UAPI documentation	Userspace-visible ioctl ordering, capability discovery, register access, run/exit contract.
3	Neoverse and Cobalt material	Implemented features, CPU identity, product-specific limits and mitigations.
4	QEMU/KVM source and tests	Implementation corroboration; never substitutes for public UAPI obligations.
5	Repository and companion paper	Claim inventory, finding taxonomy, formal audit structure.

2.3 Review procedure

Each claim was normalised as $C = (s, p, r, o, t)$: source section s , proposition p , reference r , observable evidence o , and verification method t . A claim is accepted only if its scope matches the reference and its observable is reproducible. This follows the supplied repository's Correction/Precision distinction and the uploaded excerpt's demand that quotient transitions be independent of representative choice.

2.4 Limitations

No privileged access to a live Cobalt host was supplied. No Azure VM SKU, kernel configuration, QEMU command line, firmware revision, or raw register dump was available. Consequently, this document reports specification conformance and defines tests; it does not report benchmark data or certify a deployed hypervisor.

3 Architecture Baseline and Selected Profile

3.1 Execution profile

The baseline is AArch64 with a Type-1 hypervisor executing at EL2, a guest kernel at EL1, and guest applications at EL0. The selected profile is non-VHE for specification clarity: the host control plane is not modelled as an EL2-hosted kernel. VHE is discussed only to prevent non-VHE requirements from being mistaken for universal constraints.

EL2: KVM/hypervisor control, Stage-2 tables, virtual interrupt and trap policy
 ↓ ERET / synchronous and asynchronous exception boundary
EL1: guest kernel, guest Stage-1 translation, virtual exception handling
 ↓ syscall / exception boundary
EL0: guest process state and application-visible ISA

3.2 Architectural ownership

Object	Owner in profile	Assurance consequence
HCR_EL2	EL2	Enables Stage-2 and defines major guest traps/routing.
VTTBR_EL2, VTCR_EL2	EL2	Selects translation tables, VMID, address size and walk attributes.
EL2 vector and return state	EL2	Determines exception entry, ELR_EL2/SPSR_EL2, and safe return.
Guest Stage-1 tables	EL1, constrained by EL2	Translate VA to IPA but cannot grant access absent at Stage-2.
Stage-2 tables	EL2/KVM	Translate IPA to PA and impose independent access and execute limits.
Physical interrupt controller	Host/EL2	Delivers or virtualises interrupts according to configured policy.

3.3 Profile requirements

- P-01: The implementation shall identify whether it runs VHE or non-VHE before applying profile-specific HCR_EL2 constraints.
- P-02: Guest-visible ID registers shall be derived from an explicit, stable virtual CPU model rather than blindly exposing the host.
- P-03: Host feature discovery, guest feature projection, and migration compatibility shall be recorded as separate decisions.
- P-04: Unsupported architectural features shall fail closed or be consistently hidden from the guest.

4 EL2 Hypervisor Configuration Results

4.1 Reviewed HCR_EL2 controls

Field	Value	Design meaning	Result
VM	1	Enable Stage-2 translation for EL1/EL0 accesses in the selected regime.	Conforms
RW	1	EL1 executes AArch64.	Conforms
TSC	policy	Trap selected system instructions such as SMC where required.	Verify policy
TWI/TWE	policy	Trap WFI/WFE according to scheduler and power-management design.	Verify policy
TGE	0	Route general exceptions according to non-VHE guest-at-EL1 design.	F-04
VSE	0 normally	Virtual SError injection controlled by explicit error-delivery state.	Conforms

4.2 F-04: TGE scope

The phrase “TGE=1 is not valid” is too broad. It is not used by this non-VHE design, but it is architecturally meaningful in a VHE context when considered with HCR_EL2.E2H. The corrected requirement is:

EL2-REQ-04. For the selected non-VHE profile, the implementation shall enter the guest with HCR_EL2.TGE=0. Implementations using VHE shall define and verify a separate E2H/TGE configuration and exception-routing model.

4.3 Entry consistency set

The following values form one entry transaction and shall not be treated as unrelated settings: HCR_EL2; VTTBR_EL2; VTCR_EL2; guest return state in ELR_EL2/SPSR_EL2; timer and interrupt virtualisation state; and applicable trap registers. A stale value from another vCPU can violate isolation even if each field is individually legal.

4.4 Test result specification

- EL2-T01: Read back the effective control state immediately before guest entry using a debug build or tracepoint.
- EL2-T02: Execute guest WFI/WFE, SMC, system-register, and exception-routing probes and compare exits with policy.
- EL2-T03: Switch repeatedly between vCPUs with distinct VMIDs and trap policies; detect cross-vCPU control leakage.
- EL2-T04: Run separate VHE and non-VHE configurations; prove that F-04 is profile-scoped.

5 EL2 Context, Entry, and Exit

5.1 Context set

The reviewed core list—HCR_EL2, VTTBR_EL2, VTCR_EL2, SCTLR_EL2, TCR_EL2, MAIR_EL2, SPSR_EL2, ELR_EL2, SP_EL2, and interrupt masks—is a defensible architectural inventory for the specified profile. It is not a complete KVM implementation context: feature-specific state, virtual timers, GIC state, debug, PMU, SVE/SME, pointer authentication, MTE, and nested-virtualisation controls must be included when exposed.

5.2 State-group requirements

Group	Required property	Failure effect
Return state	ELR_EL2 and SPSR_EL2 correspond to the same guest boundary.	Wrong PC, mode, masks, or stack selection.
Translation state	VMID, Stage-2 root, geometry, and TLB state are mutually consistent.	Cross-VM alias or translation fault.
Interrupt state	DAIF and virtual interrupt routing are restored as a coherent policy state.	Lost, delayed, or incorrectly delivered interrupt.
Feature state	Every guest-exposed feature has complete save/restore or an architectural reset value.	Information leak or guest corruption.
Exit record	Syndrome, fault address, exit reason, and vCPU identity remain linked.	Non-auditable handling or wrong emulation.

5.3 Exit handling sequence

1. Capture the architectural syndrome and affected guest state before destructive host handling.
2. Classify the exit as in-kernel handled, userspace-visible, injectable guest exception, or fatal vCPU error.
3. Preserve the distinction between request, emulation completion, and guest-visible completion.
4. Update only the state justified by the exit contract, then validate entry invariants before re-entry.

5.4 Result

Conforms with extension requirements. The repository’s basic save/restore proposition is sound, but a production specification must make the context set conditional on the projected virtual CPU feature set. “All EL2 state” is not a stable list across N2, V3, kernel versions, or optional features.

EL2-REQ-08. The implementation shall derive its vCPU context schema from the guest-visible feature contract and shall reject migration when the destination cannot restore an observationally equivalent state.

6 Stage-2 Translation Model

6.1 Translation composition

For a guest access, Stage-1 maps virtual address (VA) to intermediate physical address (IPA); Stage-2 maps IPA to physical address (PA). Effective access permission is no greater than the intersection of the two stages and other architectural controls. A valid Stage-1 mapping cannot bypass a missing or restrictive Stage-2 mapping.

$$VA \xrightarrow[\text{guest}]{S1} IPA \xrightarrow[\text{EL2/KVM}]{S2} PA$$

6.2 Isolation invariants

1. S2-I1: Every guest-reachable IPA resolves only to pages assigned to that VM or to intentionally shared objects.
2. S2-I2: An unmapped or permission-incompatible IPA generates a Stage-2 fault and is not silently redirected.
3. S2-I3: EL1 cannot modify VTTBR_EL2, VTCR_EL2, or the hypervisor-owned Stage-2 table pages.
4. S2-I4: Device apertures use Device memory types and explicit access bounds; normal RAM uses coherent Normal memory attributes.
5. S2-I5: A VMID identifies one live translation context at a time, or reuse is preceded by the required invalidation protocol.
6. S2-I6: Table update, cache maintenance where applicable, barriers, TLBI, and re-entry occur in the architecturally required order.

6.3 Fault evidence

For each injected fault the result record shall contain vCPU, VM, IPA/VA where available, exception class, syndrome, access type, translation level, entry generation, VMID, handling decision, and whether userspace was notified. The host shall not infer that a fault was guest-caused until the failing mapping and ownership state have been correlated.

6.4 Result

Conforms. The repository’s isolation claims are consistent with the baseline. The operational requirement is stronger than “unmapped IPA faults”: table ownership, update ordering, and fault provenance must also be demonstrated.

6.5 Negative tests

Map adjacent pages to different owners; remove read/write/execute independently; test block-to-table splits; race memslot deletion with vCPU execution; and attempt access across the highest configured IPA boundary. Each access shall either reach the intended page with intended permissions or produce a classified fault.

7 VTTBR, VTCR, and Address-Space Configuration

7.1 Control fields

Register/field	Review focus	Requirement
VTTBR_EL2.BADDR	Stage-2 root	Correct alignment and root table for the active VM.
VTTBR_EL2.VMID	TLB context tag	Match the VMID allocator generation and invalidation state.
VTCR_EL2.T0SZ	IPA input size	Encode the supported guest IPA range.
VTCR_EL2.SL0	Start level	Be consistent with granule and address geometry.
VTCR_EL2.TG0	Granule	Match table construction and host support.
IRGN0/ORGN0	Walk cacheability	Match coherent page-table memory and platform rules.
VTCR_EL2.SH0	Shareability	Match the system coherency domain.
VTCR_EL2.PS	Output PA size	Not exceed implemented physical address capability.
VTCR_EL2.VS	VMID width	Select a supported 8- or 16-bit VMID regime consistently.

7.2 Configuration algorithm

1. Discover architectural capabilities and KVM's supported IPA size; do not copy constants from a different host generation.
2. Choose a guest IPA size and granule supported by both the userspace VMM and KVM.
3. Construct Stage-2 tables with descriptors compatible with the chosen geometry.
4. Allocate a VMID and generation; associate it atomically with the root table.
5. Perform required ordering and invalidation before making the configuration executable.
6. Record the effective values as part of the VM evidence manifest.

7.3 Key result

The repository correctly identifies T0SZ/SL0 consistency and VS semantics as critical. The study adds one qualification: userspace using KVM normally supplies memory regions and a VM type/capability choice rather than directly programming every EL2 bit. Therefore, the auditable contract must link userspace configuration to the effective values selected by the kernel.

S2-REQ-10. A test build shall expose or attest the effective IPA size, Stage-2 granule, VMID width, and root-generation relationship. A userspace request alone is insufficient evidence that the effective EL2 configuration matches the specification.

8 Stage-2 Updates, VMIDs, and TLB Maintenance

8.1 Update classes

Change	Required reasoning	Principal hazard
Invalid to valid	Publish a fully initialised descriptor with required ordering.	Walker observes partial/stale entry.
Permission reduction	Ensure stale permissive translations cannot survive.	Guest retains write or execute access.
Output-address change	Apply break-before-make where required and invalidate old translation.	One IPA resolves to inconsistent PAs.
Block/table change	Respect descriptor transition and walk-cache rules.	Mixed-level aliases.
Memslot removal	Stop or synchronise vCPUs, unmap, invalidate, then release backing.	Use-after-free / cross-tenant access.
VMID reuse	Invalidate translations associated with the old context before reassignment.	Cross-VM TLB alias.

8.2 Finding F-06: evidence gap

The claim that VMID reuse without the required Stage-1-plus-Stage-2 invalidation is a correctness defect is accepted. However, a specific invocation such as `VMALLS12E1IS` must be justified against the implemented regime and kernel path, not copied as an isolated mnemonic. The proof obligation is that no translation tagged with the reused VMID can be consumed after ownership changes.

8.3 Required tests

- Force rapid VMID allocator wrap under concurrent guests; fingerprint physical pages and detect any stale resolution.
- Change an IPA mapping between two backing pages while vCPUs execute read/write loops; accept only architecturally permitted observations.
- Downgrade RWX permissions one dimension at a time and time the last permitted access relative to the update protocol.
- Delete and recreate memslots while issuing MMIO and RAM accesses at both boundaries.
- Migrate a stopped VM, reuse source resources, and prove the target never observes source-host stale translations.

8.4 Acceptance rule

Zero cross-owner translations are permitted. Translation faults during a deliberately disruptive negative test are acceptable only when classified and contained. Any successful access through a stale mapping is a release-blocking failure irrespective of its statistical frequency.

9 Memory Types, Permissions, and Fault Handling

9.1 Attribute policy

Region	Stage-2 intent	Review condition
Guest RAM	Normal, cacheable, coherent attributes	Compatible with host mapping and shareability; no unintended device alias.
Read-only image	Normal, read-only; execute policy explicit	Writes fault; executable scope is minimal and measured.
Guest page tables	Normal memory, guest writable as required	Protected from other VMs; Stage-2 execute policy justified.
Emulated MMIO	Unmapped or device aperture causing handled exit	No direct mapping to unrelated host physical memory.
Passed-through MMIO	Device-nGnRE or stricter as required	Aperture, IOMMU, interrupt, and reset isolation proven.
Shared page	Normal memory with explicit participants	Lifetime, cache coherence, and revocation protocol recorded.

9.2 F-07: verification gap

The statement that device MMIO uses nGnRE or nGnRnE is directionally correct, but the correct attribute is device- and mapping-specific. The study therefore accepts the prohibition on Normal-memory MMIO while requiring evidence for the chosen Device subtype, ordering semantics, host mapping, and IOMMU configuration.

9.3 Fault disposition

1. Decode the syndrome and establish whether the fault arose at Stage-1, Stage-2, or another permission mechanism.
2. Correlate the IPA with the current memslot and mapping generation.
3. If the access targets an emulated aperture, validate size, alignment, direction, and device state before emulation.
4. If no valid emulation exists, inject the defined guest fault or stop the vCPU; never fabricate a successful access.
5. Record enough state to reproduce the decision without retaining unrelated guest data.

9.4 Acceptance metrics

All synthetic invalid accesses must be rejected; all valid accesses must resolve to the expected backing object; no W+X mapping may appear unless explicitly approved; and every userspace MMIO exit must be traceable to one guest instruction boundary and one emulation response.

10 KVM/arm64 API Object and Capability Model

10.1 Object lifecycle

The stable KVM ABI is organised around file descriptors and ioctls. Userspace opens `/dev/kvm`, checks the API version and capabilities, creates a VM fd, configures guest memory and devices, creates vCPU fds, maps `kvm_run`, initialises architecture-specific vCPU features, and invokes `KVM_RUN`. VM, vCPU, and device operations have distinct ownership and threading expectations.

```
system_fd = open("/dev/kvm")
assert ioctl(system_fd, KVM_GET_API_VERSION) == KVM_API_VERSION
check required KVM_CAP_* extensions
vm_fd = ioctl(system_fd, KVM_CREATE_VM, vm_type)
configure VM capabilities, memory slots, irqchip and devices
vcpu_fd = ioctl(vm_fd, KVM_CREATE_VCPU, index)
mmap kvm_run; KVM_ARM_VCPU_INIT; configure registers/features
finalize features that require finalization; then KVM_RUN
```

10.2 Requirements

- KVM-01: Capability discovery shall precede any use of an optional ioctl, register group, VM type, or vCPU feature.
- KVM-02: The implementation shall record capability values, not infer availability from kernel release alone.
- KVM-03: VM ioctls shall be issued from the supported process/address-space context; vCPU execution ownership shall be explicit.
- KVM-04: File descriptor duplication, fork, and teardown semantics shall be controlled so that object lifetime matches the VM lifecycle.
- KVM-05: Every `KVM_RUN` exit shall be handled according to the returned exit reason and completion contract.
- KVM-06: Unknown capabilities, registers, and exit reasons shall fail safely and generate diagnostic evidence.

10.3 Result

Conforms. The repository correctly anchors lifecycle claims in the KVM API rather than in QEMU implementation detail. The main strengthening is to make capability evidence part of the session manifest; this avoids treating a kernel version or Cobalt generation as a substitute for runtime discovery.

11 vCPU Lifecycle State Machine

11.1 F-02: missing finalisation state

The original direct edge *configured* → *running* is invalid for an SVE-enabled arm64 vCPU that requires finalisation. The corrected state machine is:

created → **configured**: KVM_ARM_VCPU_INIT, feature/register setup
configured → **finalised**: required feature finalisers succeed
finalised → **running**: KVM_RUN enters guest
running → **vmexit**: trap, interrupt, fault, shutdown, or userspace exit
vmexit → **running**: exit completion and re-entry checks succeed
vmexit → **migrating**: state captured at a defined quiescent boundary
migrating → **finalised-equivalent**: compatible state restored on target
finalised/vmexit → **destroyed**: references closed and resources released

11.2 State invariants

State	Invariant
Created	vCPU fd exists; no assumption that architectural features are initialised.
Configured	Initialisation applied; mutable feature configuration may still be legal.
Finalised	Required feature configuration is locked; entry preconditions can be evaluated.
Running	One execution owner; guest state may change until exit.
VM exit	Exit record stable enough for the designated handler and completion path.
Migrating	Source state quiesced; schema, feature, and target compatibility checked.
Destroyed	No runnable references or externally retained mappings remain.

11.3 Acceptance tests

Attempt KVM_RUN before initialisation and before required finalisation; expect rejection. Finalise twice and attempt post-finalisation changes; verify documented error behaviour without depending on one errno unless the ABI guarantees it. Close vCPU and VM descriptors in varied orders while checking that no execution or mapping survives the last valid reference.

12 SVE/SVE2 Configuration and Finalisation

12.1 Feature discovery and state

The architectural discovery sequence first establishes SVE support using `ID_AA64PFR0_EL1.SVE`; SVE2 capability is then interpreted from `ID_AA64ZFR0_EL1`. Guest state includes 32 scalable Z registers, 16 predicate registers, FFR, and the effective vector length, plus control state required by the virtual CPU model.

12.2 F-03: selection is not locking

`KVM_SET_ONE_REG` targeting `KVM_REG_ARM64_SVE_VLS` communicates the permitted vector-length bitmap after vCPU initialisation. It does not itself create the lifetime invariant. `KVM_ARM_VCPU_FINALIZE` with the SVE feature is the transition after which the negotiated configuration is fixed for execution.

```
KVM_ARM_VCPU_INIT(vcpu, features including SVE)
GET_ONE_REG(KVM_REG_ARM64_SVE_VLS) // discover supported set
SET_ONE_REG(KVM_REG_ARM64_SVE_VLS) // select migration-safe set
KVM_ARM_VCPU_FINALIZE(KVM_ARM_VCPU_SVE) // lock configuration
GET/SET complete SVE state as permitted; then KVM_RUN
```

12.3 Corrected invariants

- SVE-I1: The selected VLS set is a subset of KVM-reported support and the migration pool contract.
- SVE-I2: VL invariance begins at successful finalisation, not at vCPU creation or the first register write.
- SVE-I3: Destination restoration shall preserve all guest-visible scalable state and an equivalent VLS contract.
- SVE-I4: Save-area sizing shall be derived from the maximum exposed VL and kernel ABI, with overflow-safe arithmetic.
- SVE-I5: Tests shall cover Z, P, FFR, FPSR/FPCR interactions, signal/exception boundaries, and repeated vCPU switches.

12.4 Result

Conforms after precision repair. The architectural state description is sound. Any mandated save/restore ordering must be tied to the relevant ABI or implementation contract; correctness is ultimately judged by complete state preservation, not by an undocumented userspace ordering assumption.

13 Migration and Observable Equivalence

13.1 Formal audit lens

The uploaded excerpt defines observational equivalence $s_1 \approx s_2$ through equality of observations and requires transition congruence for a well-defined quotient system. Applied here, two vCPU implementations may differ internally yet be migration-equivalent only if permitted guest and audit observations agree and matching future transitions remain available.

$$s_1 \approx s_2 \iff O(s_1) = O(s_2), \quad s_1 \rightarrow t_1 \Rightarrow \exists t_2 : s_2 \rightarrow t_2 \wedge t_1 \approx t_2$$

13.2 Migration observation vector

$O(s)$ shall include at least: general registers and PC/PSTATE; system registers in the guest contract; timer state; interrupt-controller state; pending exceptions; Stage-2-visible memory content and permissions; device state; SVE/SVE2 state and VLS; feature ID register values; and monotonic lifecycle/finalisation metadata. Microarchitectural cache state need not be identical, but must not create a guest-visible security or correctness divergence outside the stated model.

13.3 Compatibility gates

Gate	Acceptance condition	Failure action
CPU model	Destination can expose the same stable ID-register contract.	Reject migration.
Vector state	Selected VLS and all state are representable and finalisable.	Reject or cold restart by policy.
Memory regime	IPA size, granule, attributes, and dirty-state transfer are compatible.	Reject migration.
Devices	State schemas and completion semantics are compatible.	Reject or detach before migration.
Security controls	Destination mitigation floor is not below the VM contract.	Reject or require approved downgrade.

13.4 Result

Migration is not identity of internal host state. It is preservation of the defined observation and transition contract. This distinction makes heterogeneous N2-to-V3 claims testable: a migration is acceptable only under a deliberately projected common CPU model, not merely because both hosts execute AArch64.

14 Lean Formal Proof Results

14.1 Proof boundary and artefact

The companion artefact `FormalResults.lean` formalises the equations from `prism-uploads/Excerpt_arXiv.pdf` and three deliberately small ARM/KVM models. It uses no unproved axiom, sorry, or host-specific constant. The logical boundary is explicit:

- **Kernel propositions:** equality, preorder closure, transition matching, permission conjunction, and finite lifecycle transitions.
- **Specification premises:** the architectural meanings assigned to Stage-2, HCR_EL2, SVE, and the KVM ioctls.
- **Measured premises:** actual ID registers, IPA width, VLS bitmap, firmware state, QEMU/kernel revision, and Azure SKU identity.

Consequently, Lean can reject an invalid deduction from the model, but hardware and ABI tests must establish that the model is instantiated by the system under review.

14.2 Excerpt theorem ledger

Result	Lean declaration	Formal conclusion
L-01	<code>transition_congruence</code>	Equivalent sources admit matching transitions to equivalent targets.
L-02	<code>refinement_reflexive</code>	Reflexivity of observation order pulls back to state refinement.
L-03	<code>refinement_transitive</code>	Transitivity of observation order pulls back to state refinement.
L-04	<code>refinement_preorder</code>	The pulled-back relation satisfies both preorder obligations.
L-05	<code>embedding_preserves_class</code>	Observation-preserving embeddings select the same extensional equivalence class.
L-06	<code>migration_identity</code>	Observation-preserving migration is identity at quotient-visible level.
L-07	<code>quotient_step_well_defined</code>	Symmetry of observation equality converts forward congruence into bidirectional representative independence.
L-08	<code>quotient_closure</code>	The excerpt's closure assumptions jointly yield refinement, embedding, migration, and transition results.

14.3 Formal statement of the closure result

Let $O : S \rightarrow \Omega$, $s \approx t \iff O(s) = O(t)$, and $s \sqsubseteq t \iff O(s) \leq O(t)$. The encoded theorem consumes, rather than invents, four assumptions:

$$\begin{aligned} &\text{Congruent}(\rightarrow, \approx), & (\Omega, \leq) \text{ is a preorder,} \\ &O(\iota(s)) = O(s), & O(\text{mig}(s)) = O(s). \end{aligned}$$

It returns refinement reflexivity and transitivity, equality of the observation fibres of s , $\iota(s)$, and $\text{mig}(s)$, plus representative-independent transition matching. The proof strengthens the prose excerpt by deriving the reverse matching direction from symmetry of equality rather than leaving representative independence informal.

```
def ObsEq (s t : State) : Prop := observe s = observe t
def Refines (leObs) (s t : State) : Prop :=
  leObs (observe s) (observe t)
def TransitionCongruence (step) : Prop :=
  forall {s1 s2 t1}, ObsEq observe s1 s2 -> step s1 t1 ->
    exists t2, step s2 t2 /\ ObsEq observe t1 t2
```

14.4 ARM and KVM proof instances

Model	Theorem	Result
Stage-2 permission	<code>stage2_denial_dominates</code>	If effective permission is $S1 \wedge S2$, a Stage-1 allow cannot bypass a Stage-2 deny.
Permission completeness	<code>access_requires_both</code>	Effective access implies both stages allow in the Boolean abstraction.
EL2 profile	<code>nonVHE_requires_tge_zero</code>	The selected non-VHE profile entails $TGE=0$.
EL2 countermodel	<code>tge_zero_is_not_universal</code>	A VHE state with $TGE=1$ is admitted by the scoped model, refuting a universal zero requirement.
KVM lifecycle	<code>run_requires_finalised</code>	Every modelled run edge starts in <i>finalised</i> .
Negative lifecycle	<code>configured_cannot_run</code>	There is no direct configured-to-running KVM_RUN edge.
SVE locking	<code>vls_immutable_after_finalisation</code>	No VLS-selection edge leaves the finalised state.
Migration restore	<code>migration_restore_is_finalised</code>	A successful restore returns the target to a finalised-equivalent phase.

14.5 Machine-check protocol

The reproducible check is a single command in a Lean 4 environment:

```
lean FormalResults.lean
```

Acceptance requires exit status zero and no occurrence of `sorry`, `axiom`, or `admit`. The present Prism runtime does not provide a Lean executable, so the source artefact and PDF are generated here while the Lean command remains a required CI/reviewer gate. This limitation is recorded rather than representing an unexecuted proof as a kernel certificate.

14.6 Primary-source formula tests

Source	Tested formula/model	Disposition
Arm DDI0487K	$VA \xrightarrow{S1} IPA \xrightarrow{S2} PA$; effective restrictions; EL2-owned translation controls	Model consequence proved; descriptor legality, TLBI, barriers, and faults require architectural tests.
Arm DDI0608A	Armv9 additions used by N2/V3, including SVE2-facing feature interpretation	Feature-state claims remain conditional on ID-register evidence.
Neoverse N2 TRM r0p1	N2 implementation baseline and implemented feature set	Supports the Armv9/N2 classification; exact host exposure remains measured.
Neoverse V3/CSS V3	V3 Armv9.2-A class and subsystem implementation claims	Source-consistent; Cobalt 200 mapping remains an SKU verification gap.
Linux KVM/arm64 API	init, VLS selection, SVE finalisation, run, and post-finalisation immutability	Lifecycle model matches the documented ioctl ordering and rejection boundary.
QEMU target/arm/	Host feature/VLS query and <code>KVM_ARM_VCPU_FINALIZE</code> invocation	Independent implementation witness; not a substitute for the KVM ABI.
Companion excerpt	Quotient congruence, refinement, embeddings, and migration identity	All stated structural derivations are represented in the Lean artefact.

15 Cobalt N2/V3 Substrate Assessment

15.1 F-01: Cobalt 100 ISA baseline

The repository identifies a factual error: Cobalt 100 is associated with the Neoverse N2 lineage and must not be labelled Armv8.5-A. The corrected architectural baseline is **Armv9.0-A**. Compiler `-march` profiles, marketing feature lists, and architectural version names are not interchangeable evidence.

Platform	Microarchitecture	Study baseline	Disposition
Cobalt 100	Neoverse N2 lineage	Armv9.0-A	Correct F-01 ; verify host MIDR/features.
Cobalt 200	Neoverse V3/CSS V3 lineage	Armv9.2-A class claim	Verification gap F-05; confirm exact SKU and exposure.

15.2 Portability boundary

N2 and V3 are not interchangeable execution substrates. Differences may include implemented vector length, ID-register fields, pointer-authentication algorithm exposure, MTE level, RME/CCA support, branch-record controls, PMU/debug facilities, mitigation mechanisms, and performance behaviour. A common guest model must be the stable intersection intentionally exposed by KVM/QEMU, not a raw intersection guessed from product names.

15.3 Host evidence manifest

For each tested Azure SKU retain: VM SKU and region; host CPU implementer/part/revision where exposed; kernel and KVM version; firmware/microcode identifiers available to the tenant; QEMU/VMM version and machine type; KVM_CHECK_EXTENSION results; guest ID-register dump; SVE VLS bitmap; IPA size; page granule; mitigation status; and the exact launch configuration.

15.4 Results specification

- CBLT-T01: Establish the CPU-model contract independently on N2 and V3 hosts.
- CBLT-T02: Boot identical guests under the common model and compare architectural observations.
- CBLT-T03: Perform stopped-state migration both directions if the service permits; otherwise replay exported state through an equivalent laboratory pair.
- CBLT-T04: Reject any feature whose destination representation or mitigation floor is weaker than the declared contract.

16 Security and Microarchitectural Review

16.1 Threat boundary

The study distinguishes architectural isolation from microarchitectural non-interference. Correct Stage-2 translation prevents architectural reads of unassigned memory; it does not alone eliminate branch-predictor, cache, TLB, speculative-execution, PMU/debug, or shared-resource channels.

Threat	Control family	Evidence required
Branch target/history injection	CSV2-class mechanisms, BHB clearing where applicable, entry sequencing	Host feature and kernel mitigation status; adversarial cross-vCPU test.
Return prediction leakage	Architecture/implementation-specific return-stack handling	Kernel path review and host-generation evidence.
Cache timing	Scheduling/isolation policy, cache partitioning where available, constant-time guest code	Measured leakage model and tenant threat assumptions.
Speculative EL2 data exposure	Minimal mappings, permission discipline, patched firmware/kernel	Mapping audit, versions, known-issue assessment.
Debug/PMU/BRBE leakage	Trap, disable, sanitise, or virtualise state	Entry/exit tests and complete state schema.
MTE/PAC differences	Stable guest projection and complete context handling	Cross-host ID register and state migration tests.

16.2 F-08: verification gap

The repository's threat descriptions and proposed control families are suitable as a review checklist, but feature presence does not prove correct enablement. For example, an ID-register indication, a kernel mitigation string, and an effective context-switch sequence are different evidence objects. Each must be captured separately.

16.3 Security requirements

- SEC-01: No guest shall receive direct access to host EL2 mappings, hypervisor table pages, or another VM's physical pages.
- SEC-02: Debug, PMU, trace, and branch-record features shall default to trapped or sanitised unless explicitly virtualised.
- SEC-03: The migration compatibility floor shall include required security mitigations, not ISA functionality alone.
- SEC-04: Failure to establish mitigation state shall be reported as unknown, never silently converted to compliant.
- SEC-05: Cryptographic workloads shall be assessed for constant-time behaviour independently of hypervisor isolation.

17 Verification Matrix and Result Schema

17.1 Priority test catalogue

ID	Objective	Acceptance result	Link
T-01	EL2 profile	Effective VHE/non-VHE state matches scoped controls.	F-04
T-02	Stage-2 isolation	Invalid IPA and permissions always fault; no cross-owner mapping.	S2-I1-6
T-03	VMID reuse	No stale translation after allocator wrap/reassignment.	F-06
T-04	Attribute map	RAM/MMIO types and apertures match approved device map.	F-07
T-05	Lifecycle negatives	Run-before-init/finalise and illegal post-finalise writes are rejected.	F-02/03
T-06	SVE state	All Z/P/FFR and control observations survive switches and exits.	F-03
T-07	Migration	Source and target satisfy observation/transition equivalence.	MIG
T-08	Cobalt identity	SKU, MIDR, ID registers and capabilities support platform claim.	F-01/05
T-09	Exit handling	Every exercised exit has correct handler and completion semantics.	KVM-05
T-10	Side channels	Mitigation status is known; residual tests meet approved threshold.	F-08

17.2 Auditable result object

Each result shall be serialised with the following minimum fields: `resultId`, `claimId`, `testId`, baseline document and section, repository revision, host manifest hash, VMM/kernel build identifiers, preconditions, stimulus, expected observation, actual observation, disposition, reviewer, timestamp, and hashes of logs or register dumps. Corrections require retest; Precision findings require an amended scoped requirement and test.

17.3 Exit criteria

1. F-01 through F-04 are incorporated into the normative specification and regression tests.
2. All Correction findings are closed with passing evidence; no open Correction may be accepted as residual risk.
3. Verification gaps affecting isolation, lifecycle legality, or migration are closed for every supported host class.
4. The guest CPU model and migration pool are versioned and immutable for a running VM.
5. Test artefacts permit an independent reviewer to reproduce each result or explain environmental limits.

17.4 Decision authority

Architecture review accepts the register and transition model; hypervisor engineering accepts implementation evidence; platform engineering accepts Cobalt host claims; security accepts mitigation evidence and residual risk. A single successful boot is not an acceptance result.

18 Conclusions, Actions, and References

18.1 Conclusion

The supplied quotient and refinement formulae have been converted into explicit Lean declarations together with small Stage-2, EL2-profile, and KVM lifecycle proof models. The model results support the corrected engineering specification: Stage-2 denial dominates Stage-1 permission, $TGE=0$ is profile-scoped, and SVE configuration, finalisation, and execution are separate states. A Lean kernel run is still required because this Prism runtime has no Lean executable. Independently, Cobalt family names cannot replace architectural and runtime capability evidence.

18.2 Required actions

1. Correct Cobalt 100 to Armv9.0-A / Neoverse N2 and attach SKU-level evidence.
2. Insert *finalised* into the normative vCPU state machine and migration restoration model.
3. Attribute SVE VLS immutability to successful `KVM_ARM_VCPU_FINALIZE`, not `SET_ONE_REG`.
4. Scope `HCR_EL2.TGE=0` to the non-VHE profile and define a separate VHE profile if required.
5. Run `lean FormalResults.lean` in CI, require zero errors and prohibit `sorry/axiom/admit`.
6. Implement the verification matrix and retain auditable result objects.

18.3 Primary references

1. Arm Ltd., *Arm Architecture Reference Manual for A-profile architecture*, DDI0487K; pin section and rule identifiers in each claim record.
2. Arm Ltd., *Arm Architecture Reference Manual Supplement*, Armv9-A, DDI0608A.
3. Arm Ltd., *Arm Neoverse N2 Core Technical Reference Manual*, document 102099, r0p1/r0p1-equivalent revision used by the test target.
4. Arm Ltd., *Arm Neoverse V3 Core Technical Reference Manual*, document 107734, and applicable CSS V3 materials.
5. Linux kernel documentation, *The definitive KVM API documentation*, especially `KVM_ARM_VCPU_INIT`, `KVM_REG_ARM64_SVE_VLS`, `KVM_ARM_VCPU_FINALIZE`, and `KVM_RUN`: <https://docs.kernel.org/virt/kvm/api.html>.
6. QEMU project, ARM target and KVM implementation, `target/arm/`, with the reviewed commit pinned in the evidence record: <https://gitlab.com/qemu-project/qemu/-/tree/master/target/arm>.
7. Supplied companion excerpt, *Structural Closure and Quotient Consistency*, `prism-uploads/Excerpt_arXiv.pdf`, pp. 1–4; formalisation in `FormalResults.lean`.
8. Microsoft Azure Cobalt 100/200 platform material and host/SKU evidence applicable to the deployment under test.

18.4 Final disposition

FORMALISED; KERNEL CHECK AND PLATFORM EVIDENCE PENDING. The proof source represents the excerpt’s closure results and the corrected ARM/KVM models without admitted propositions. Claim “Lean verified” becomes final only after a recorded zero-error Lean run. Production, migration, and security claims remain unapproved until F-05–F-08 are closed on the actual Cobalt host classes.